

Consist: Provenance-Aware Caching and Queryable Provenance for Scientific Workflows

Zachary A. Needell¹¶

¹ Lawrence Berkeley National Laboratory, United States  ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Adithi R Upadhy](#) 

Submitted: 17 June 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

Computational scientific workflows produce outputs that are functions of three things: the code used to produce them, the configuration settings that parameterize their behavior, and the set of input objects they consume. Each of those inputs may itself be the output of an earlier computational step, creating dependency chains that can extend across model runs. In practice, these chains are rarely explicit: researchers frequently cannot trace which exact code, configuration, and upstream inputs produced a given result, or which downstream outputs are affected by an upstream change.

Consist is a Python library that makes this lineage explicit, persistent, and queryable. It wraps ordinary Python computations in a provenance layer that records which code, configuration, and input artifacts produced each output, computing a deterministic identity signature. Each step's identity incorporates the identities of its inputs, including the signatures of the runs that produced them. As a result, any output can be traced back through its full chain of upstream dependencies to the original inputs and assumptions at the system boundary. A consequence of this identity-based lineage is correct, automatic caching: upstream changes invalidate only the downstream steps that genuinely depend on them, and unchanged steps are safely reused without re-execution.

Provenance is persisted in a local DuckDB database ([Raasveldt & Mühleisen, 2019](#)) that also serves as an analytical engine: users can query across run outputs—whether ingested or remaining distributed on disk—so that cross-scenario comparison, lineage tracing, and result recovery are available through standard SQL. The entire stack is self-contained with no external services, reducing barriers to provenance tracking in research settings where simplicity of deployment matters.

Statement of Need

Many scientific domains rely on workflows where multiple models interact iteratively: climate projections couple atmosphere and ocean models, energy system analyses link capacity expansion with dispatch simulation, and epidemiological studies chain population dynamics with intervention models. In transportation planning—the domain that motivated Consist—land-use models produce population and employment forecasts; vehicle ownership models determine household fleets; activity-based demand models generate travel behavior; and network assignment models evaluate congestion and accessibility. These models run iteratively across multi-year horizons under multiple policy scenarios, with congestion feeding back into demand and demand feeding back into land use.

These workflows create four recurring problems. First, **provenance ambiguity**: when a result changes between runs, it is often unclear which upstream input, configuration parameter, or code change caused the difference. Second, **redundant recomputation**: extending a converged

10-iteration equilibrium run to 15 iterations should not require re-executing the first 10, yet without explicit dependency tracking and state management it typically does. Third, **scattered outputs**: results accumulate as directories of files with ad hoc naming, making systematic comparison across scenarios a manual exercise. Fourth, **implicit wiring**: dependencies between steps are encoded through file paths, global variables, or framework conventions rather than in inspectable, queryable structure. Consist addresses these problems by treating each workflow step as a provenance-aware computation with explicit identity and lineage.

State of the Field

Existing tools address fragments of this problem. The closest analogue in scientific computing is Snakemake (Mölder et al., 2021), which provides directed acyclic graph (DAG)-based execution with file-content-based recomputation. However, Snakemake requires expressing workflows in its own rule-based domain-specific language (DSL) rather than wrapping existing Python code, and its focus is correct re-execution rather than a persistent, queryable provenance store. Workflow engines such as Nextflow (Di Tommaso et al., 2017) similarly emphasize orchestration, requiring specific execution models without providing post-hoc queryable provenance. Earlier Python-native provenance tools such as Sumatra (Davison, 2012) and Recipy (Jackson et al., 2018) track runs with minimal code changes but lack identity-based caching, multi-step lineage propagation, and queryable output stores. DVC (Iterative, 2020) versions data files and pipelines but targets data-asset management rather than run-level identity and experiment history, while MLflow (Chen et al., 2020) offers rich experiment tracking oriented toward machine-learning training loops rather than multi-step simulations with heterogeneous configuration ecosystems. OpenLineage (OpenLineage contributors, 2026) standardizes lineage event emission but does not itself persist provenance or use lineage for execution decisions; Consist can optionally emit OpenLineage events while natively handling provenance storage, querying, and identity-based cache resolution.

Apache Hamilton (Krawczyk & Izzy, 2022) is the closest existing tool: it builds function-level DAGs with caching and Merkle-style identity propagation. However, Hamilton requires its steps to be pure Python functions, a major barrier for scientific workflows that rely on containers and subprocesses. It also lacks support for the complex configuration structures common in simulation pipelines and provides no analytical engine to make provenance metadata and artifact contents queryable.

Consist fills this gap by combining non-invasive provenance tracking, deterministic identity-based caching with lineage propagation, and a queryable analytical surface for post-hoc comparison—without requiring a workflow DSL or external services.

Software Design

The Identity Model

Consist's core mechanism is a deterministic three-part signature computed for each computational step: `signature = SHA256(code_hash || config_hash || input_hash)`. The code hash captures relevant implementation state (configurable from full-repository Git state, callable-scoped source code, or container manifest). The config hash captures a canonicalized dictionary of model settings. The input hash captures the identity of declared input artifacts, and when those artifacts were themselves produced by Consist, their identity incorporates the producing run's signature, inducing a Merkle-style DAG over the workflow. A change to any upstream step propagates invalidation only through genuinely dependent downstream computations (Figure 1).

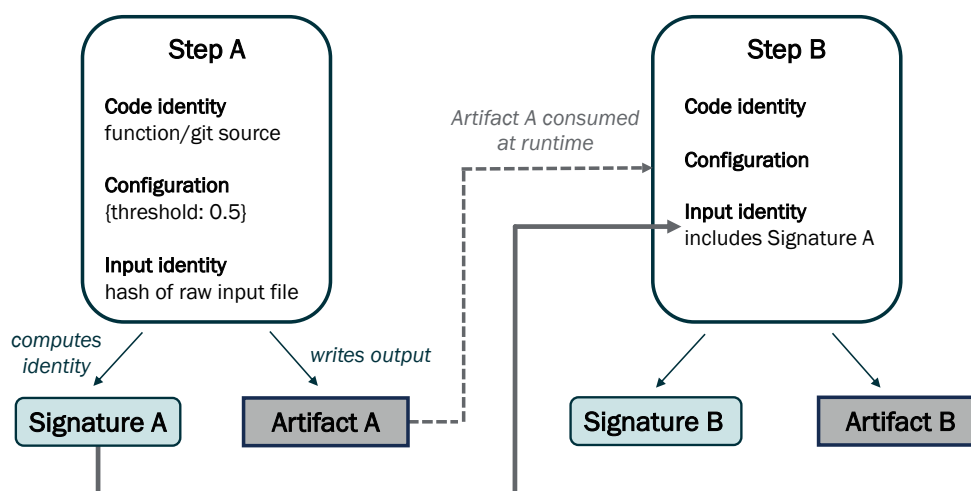


Figure 1: Figure 1. Identity propagation across workflow steps. Each step's signature is computed from its code identity, configuration, and input identity. When Step B consumes an artifact produced by Step A, Signature A is incorporated into Step B's input identity (solid arrow), propagating cache invalidation through the workflow. The artifact's bytes are consumed separately at runtime (dashed arrow) and need not be materialized when a cache hit makes re-execution unnecessary.

Why This Is Hard in Practice

Real workflows complicate this clean conceptual model.

Structured configurations: In complex simulation ecosystems, “configuration” simultaneously plays several roles: parameters that define experimental identity and must trigger recomputation when changed; metadata useful for later filtering and grouping but irrelevant to execution semantics; and large external file trees (YAML, CSV, XML) that collectively define model behavior but are impractical to store as queryable structured metadata. Consist separates these explicitly into identity-bearing configuration (structured parameters that are both hashed for cache identity and stored as queryable metadata), facets (indexed for analytical queries but not part of cache identity), and identity inputs (files or directories whose content is hashed for cache identity but not stored as queryable metadata).

Functions with side effects: Domain models write files, call subprocesses, and interact with external tools that cannot be refactored into pure functions. Consist tracks such processes through their declared inputs, configuration, and outputs without requiring domain code to become aware of the provenance engine. This motivates a design contrast from pure-function frameworks—Consist artifacts live on disk rather than in memory, and Consist loads or attaches them to steps only as required. This better accommodates outputs too large to hold in memory, which would otherwise demand manual serialization logic at every step boundary. It also enables a distinction between “hydration” (recovering a cached result's metadata and provenance) and “materialization” (ensuring its bytes exist on disk), so a cache hit can resolve cheaply without reading or duplicating files at all. In a multi-step workflow where only a downstream parameter has changed, the entire upstream chain can be validated and reused without copying or even reading the intermediate files. This correctness, however, depends on inputs and code changes being faithfully declared; undeclared dependencies (e.g., untracked files, environment state, or non-deterministic computation) can produce stale cache hits.

Post-run analysis: Consist persists provenance in both JSON snapshots and a DuckDB database for analytical querying. Selected tabular outputs can be ingested alongside provenance metadata, while large or infrequently queried outputs remain file-backed. Users can query across both through a unified analytical surface with provenance columns (run ID, scenario, year, iteration)

115 attached for grouping and filtering. Consequently, agent-level trip records and population
116 tables—millions of rows stored as distributed Parquet files—can be queried through a single
117 SQL surface, enabling analyses such as regional vehicle miles traveled trends by home geography
118 across policy scenarios without loading full datasets into memory.

119 Graduated Formalization

120 At a minimum, Consist requires users to declare the inputs, outputs, and identity-bearing
121 configuration of each step—typically a few additional lines per function call. This declaration
122 makes dependency structure explicit and creates a legible record of what each step consumes and
123 produces. Beyond this minimum, users can optionally add facets for analytical filtering, register
124 configuration adapters that canonicalize complex domain configuration (current adapters exist
125 for ActivitySim (ActivitySim Consortium, 2026) and BEAM (Laarabi et al., 2024)), and define
126 typed schemas for important artifacts. Each layer requires additional upfront effort but delivers
127 proportionally richer capabilities, from basic cache reuse through queryable provenance to
128 memory-efficient typed analytical views over large simulation outputs.

129 Research Impact Statement

130 Consist was developed to support PILATES, a workflow runtime used in the BEAM CORE
131 modeling ecosystem at Lawrence Berkeley National Lab (Spurlock et al., 2024). PILATES
132 coordinates UrbanSim (Waddell, 2002) (land use), ATLAS (Jin et al., 2024) (vehicle ownership),
133 ActivitySim (ActivitySim Consortium, 2026) (travel demand), and BEAM (Laarabi et al., 2024)
134 (network assignment) across multi-year, multi-iteration scenario lifecycles. PILATES is currently
135 used in DOE-funded scenario analyses of vehicle technology change, curb utilization, and
136 supply-chain optimization.

137 During PILATES integration, adding Consist required making model-step input and output
138 bindings explicit. That process surfaced a latent cross-model dependency error: ActivitySim's
139 population-source input was bound to a valid but wrong-year UrbanSim artifact, using the
140 current-year datastore rather than the forecast-year population snapshot produced after ATLAS
141 updates. The issue was difficult to detect from individual files alone because each artifact was
142 internally valid; the lineage graph made the semantic mismatch between upstream artifact
143 roles visible.

144 Consist's caching also delivers practical savings: when ActivitySim or BEAM configurations
145 vary across scenarios, unchanged upstream UrbanSim and ATLAS steps are identified by their
146 signatures and reused rather than re-executed, typically saving several HPC node-hours per
147 cache hit. The repository includes five example notebooks demonstrating these mechanics in
148 self-contained workflows, covering cache behavior, scenario comparison, parameter sweeps,
149 iterative extension, and cross-run analysis.

150 Availability

151 Consist is available on PyPI (`pip install consist`) and developed openly at [https://github.com/LBNL-](https://github.com/LBNL-UCB-STI/consist)
152 [UCB-STI/consist](https://github.com/LBNL-UCB-STI/consist) under a BSD 3-Clause license. Documentation including a quickstart, usage
153 guide, concept guides, API documentation, and the example notebook gallery is hosted at
154 <https://lbnl-ucb-sti.github.io/consist/latest/>. The repository includes an automated test suite
155 run via continuous integration, contribution guidelines (CONTRIBUTING.md), and a code of
156 conduct. Consist requires Python 3.11+ and depends on DuckDB, SQLAlchemy, pandas, and
157 PyArrow, with optional dependencies for spatial formats, xarray-based matrix views, and data
158 ingestion via dlt.

AI Usage Disclosure

Anthropic Claude Opus 4.5-4.7 and OpenAI ChatGPT 5.2-5.5 used across software development (code generation, refactoring, test scaffolding), documentation drafting, and paper editing. The authors reviewed, edited, and validated all AI-assisted text and code; AI-assisted software components were validated through automated tests and real-world use in the PILATES integration for funded analysis work. All architectural decisions, technical claims, and domain-specific content reflect the authors' direct experience developing and deploying Consist.

Acknowledgements

This paper and the work described were sponsored by the U.S. Department of Energy (DOE) Vehicle Technologies Office (VTO) under the Systems and Modeling for Accelerated Research in Transportation (SMART) Mobility Laboratory Consortium, an initiative of the Energy Efficient Mobility Systems (EEMS) Program. Lawrence Berkeley National Laboratory operates under DOE Contract No.DE-AC02-05CH11231. The views expressed in the article do not necessarily represent the views of the DOE or the U.S. Government. The U.S. Government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this work, or allow others to do so, for U.S. Government purposes.

References

- ActivitySim Consortium. (2026). *ActivitySim: An open platform for activity-based travel modeling*. Software documentation. <https://activitysim.github.io/activitysim/>
- Chen, A., Chow, A., Davidson, A., DCunha, A., Ghodsi, A., Hong, S. A., Konwinski, A., Mewald, C., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Singh, A., Xie, F., Zaharia, M., Zang, R., Zheng, J., & Zumar, C. (2020). Developments in MLflow: A system to accelerate the machine learning lifecycle. *Proceedings of the Fourth Workshop on Data Management for End-to-End Machine Learning*. <https://doi.org/10.1145/3399579.3399867>
- Davison, A. (2012). Automated capture of experiment context for easier reproducibility in computational research. *Computing in Science & Engineering*, 14(4), 48–56. <https://doi.org/10.1109/MCSE.2012.41>
- Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319. <https://doi.org/10.1038/nbt.3820>
- Iterative. (2020). *DVC: Data version control – git for data and models*. Software documentation. <https://dvc.org/>
- Jackson, M., Wilson, R., Zwaan, J. van der, Steinbrook, D. W., Rathgeber, F., Alegre, R., Edwards, T., Costello, C., Danny, S., Chwiejczak, J., Cottaar, M., Martinez-Ortiz, C., Viegla, D., Kucharavy, A., Ruben, G., & Upadhyay, U. (2018). *Recipy* (Version v0.3.1). <https://github.com/recipy/recipy>
- Jin, L., Jackson, C. P., Wang, Y., Chen, Q., Ho, T., Spurlock, C. A., Brooker, A., Holden, J., Gonder, J., Bouzaghane, M. A., & others. (2024). Technology progress and clean vehicle policies on fleet turnover and equity: Insights from household vehicle fleet micro-simulations with ATLAS. *Transportation Planning and Technology*, 47(8), 1399–1422. <https://doi.org/10.1080/03081060.2024.2353784>
- Krawczyk, S., & Izzy, E. ben. (2022). Hamilton: A modular open source declarative paradigm for high level modeling of dataflows. In S. R. Valluri & M. Zait (Eds.), *1st international workshop on composable data management systems, CDMS@VLDB 2022, sydney, australia*,

- 203 september 9, 2022. [https://cdmsworkshop.github.io/2022/Proceedings/ShortPapers/](https://cdmsworkshop.github.io/2022/Proceedings/ShortPapers/Paper6_StefanKrawczyk.pdf)
204 [Paper6_StefanKrawczyk.pdf](https://cdmsworkshop.github.io/2022/Proceedings/ShortPapers/Paper6_StefanKrawczyk.pdf)
- 205 Laarabi, H., Needell, Z., Waraich, R., Poliziani, C., & Wenzel, T. P. (2024). *A modeling*
206 *framework for behavior, energy, autonomy and mobility (BEAM)*. Lawrence Berkeley
207 National Laboratory. [https://transportation.lbl.gov/publications/modeling-framework-](https://transportation.lbl.gov/publications/modeling-framework-behavior-energy)
208 [behavior-energy](https://transportation.lbl.gov/publications/modeling-framework-behavior-energy)
- 209 Mölder, F., Jablonski, K. P., Letcher, B., Hall, M. B., Tomkins-Tinch, C. H., Sochat, V., Forster,
210 J., Lee, S., Twardziok, S. O., Kanitz, A., Wilm, A., Holtgrewe, M., Rahmann, S., Nahnsen,
211 S., & Köster, J. (2021). Sustainable data analysis with Snakemake. *F1000Research*, 10,
212 33. <https://doi.org/10.12688/f1000research.29032.2>
- 213 OpenLineage contributors. (2026). *OpenLineage: An open framework for data lineage*
214 *collection and analysis*. Software documentation. <https://openlineage.io/>
- 215 Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database.
216 *Proceedings of the 2019 International Conference on Management of Data*, 1981–1984.
217 <https://doi.org/10.1145/3299869.3320212>
- 218 Spurlock, C. A., Bouzaghrane, M. A., Brooker, A., Caicedo, J., Gonder, J., Holden, J., Jeong, K.,
219 Jin, L., Laarabi, H., Needell, Z., & others. (2024). *Behavior, energy, autonomy & mobility*
220 *comprehensive regional evaluator: Overview, calibration and validation summary of an*
221 *agent-based integrated regional transportation modeling workflow*. [https://transportation.](https://transportation.lbl.gov/publications/behavior-energy-autonomy-mobility)
222 [lbl.gov/publications/behavior-energy-autonomy-mobility](https://transportation.lbl.gov/publications/behavior-energy-autonomy-mobility)
- 223 Waddell, P. (2002). UrbanSim: Modeling urban development for land use, transportation, and
224 environmental planning. *Journal of the American Planning Association*, 68(3), 297–314.
225 <https://doi.org/10.1080/01944360208976274>